

Session 7 · Deck 7.A1

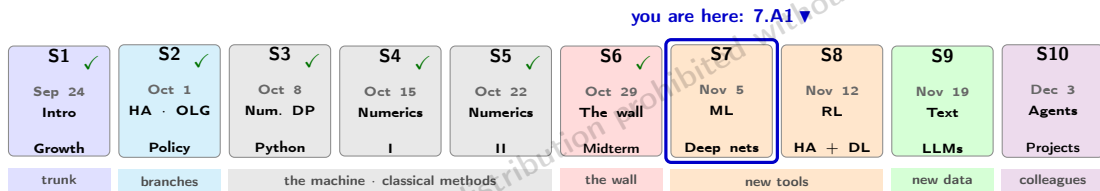
Machine Learning for Macroeconomists

Zhigang Feng

Thursday, November 5, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 7.A1 ML for macroeconomists · 7.A2 nets as approximators · 7.A3 training · 7.A4 architecture.

Session 7 in One Map

One session, two blocks, one goal: make a neural network a working instrument for a macroeconomist — why it matters, what it is, how it learns, and how to make it respect economics.

	Deck	What it settles
7.A1	ML for macroeconomists	<i>why</i> — where classical methods stall, and what a learned representation is
7.A2	Nets as approximators	<i>what</i> — neurons, layers, activations, and what the theorems do and do not promise
7.A3	Training	<i>how</i> — backpropagation, first-order optimizers, overfitting
7.A4	Architecture	● <i>which</i> — a specialized network is a constraint on W
7.B1	Growth via Euler residuals	the model becomes a loss function, and the RBC model is solved a fifth way
7.B2	Deep VFI and actor-critic	value and policy networks; where this becomes reinforcement learning
7.B3	Structure and validation	shape restrictions by construction, and how you know it worked

Four Methods, One Habit

Session 5 closed with one model solved four ways — grid VFI, first- and second-order perturbation, Chebyshev collocation. They disagree about almost everything, and they share one habit:

the approximating family is chosen before the problem is looked at. Grid points, a Taylor point, a polynomial degree. Each is a commitment made in advance, and each is why the cost explodes with the number of state variables: a tensor grid of n points per axis in d dimensions is n^d .

Smolyak (5.B3) delays that. It does not remove it.

Ledger param_count, tensor_grid_points: at $d = 20$ even three points per axis is $3^{20} \approx 3.5 \times 10^9$. Code: [s07_generate.py](#), run_param_count.

The question for this session

What if the family were **chosen by the data** — the approximant deciding for itself where to bend, and how?

That is the whole of Block A. Block B then asks the harder question: what does it take to train one on an *economic model* rather than on a dataset?

Three Things “Machine Learning” Can Mean

The phrase covers three jobs, and confusing them is the commonest mistake an economist makes with it.

1. **Prediction.** $\hat{y}$ out of sample, from features that may be text, images, or ten thousand covariates. Accuracy is the objective; interpretation is not.
2. **Causation.** Flexible nuisance functions inside an estimator that still targets a parameter — double machine learning, heterogeneous treatment effects. ML is the first stage, never the answer.
3. **Computation.** A flexible function *class* used to represent an unknown **policy or value function**, with the equilibrium conditions as the objective. **No data at all.**

This course means the third

Sessions 7–8 use networks as **solution methods**. The loss is an Euler residual, not a fit to observations; the “training set” is a set of *states you sampled*, and you may draw as many as you like.

S9 uses the first two. And note that over-fitting means something different in each: in (3) there is no sampling noise to fit — but a residual can still be driven to zero *where you sampled* and nowhere else.

Door 1: Unstructured Data

Economics has run on structured, tabular data. A great deal of economic information is not in that form — it is text, images, satellite rasters — and until recently it was simply unavailable.

- Sentiment and uncertainty from news and social media;
- satellite imagery for activity where statistics are thin — night-time lights as a GDP proxy, ships in port as trade;
- image recognition on product photographs, for goods whose features are not standardised.

Deep learning is what turns these objects into something a regression can take.

The move, in one line

An unstructured object → a **feature vector** → an ordinary economic model. 7.A4

shows this is literally how a convolutional network is built: a learned feature extractor with a feedforward network bolted on.

The measurement question does not go away. A generated regressor has sampling error of its own — S9.B1.

Door 2: Text

Natural-language processing goes past word counts, and macroeconomics has a great deal of text that matters:

- the hawkish or dovish stance of a central banker, from public communication;
- risk factors named in corporate annual reports;
- economic policy uncertainty, tracked through newspapers.

FOMC minutes, Beige Books and congressional records carry information that conventional econometrics cannot easily reach.

This is **Session 9**, in full: words to vectors, attention and the Transformer, and language models used as measurement instruments.

The caution there is the same one: a measurement produced by a model is only as stable as the model version that produced it.

Door 3: Causal Inference, Forecasting, Nowcasting

Causal inference.

- *Double machine learning* admits high-dimensional controls flexibly, cutting omitted-variable bias — nuisance functions by ML, the parameter still the target.
- ML recovers **heterogeneous** treatment effects: not the average impact, but how it varies.

Forecasting and nowcasting.

- nonlinear relations linear time-series models cannot express;
- regularisation makes a kitchen-sink regression respectable;
- real-time indicators from cards, search, imagery.

A boundary worth stating

The first stage of a DML estimator does **not** need to be differentiable — gradient boosting is usually better there, and often beats a network on tabular data. Differentiability

starts to matter only when *equilibrium conditions* become a loss and the approximant is differentiated — Block B. (The rate that matters is the $n^{-1/4}$ condition on nuisance estimates; 7.A2 says where a network's rate comes from.)

Door 4: Solving Large Models — the One This Course Walks Through

Models at the frontier — heterogeneous agents, portfolio choice over a life cycle, optimal policy without commitment — share two features:

- a high-dimensional state and genuinely nonlinear dynamics;
- grid-based methods that hit the curse of dimensionality before the model is interesting.

Two new instruments:

- **deep learning** — parameterise the policy or value function by a network (7.B1–7.B3);
- **reinforcement learning** — solve the sequential problem by simulation (S8).

Where it has been done

Optimal fiscal and monetary policy with heterogeneous agents; life-cycle portfolio choice under realistic income risk; **Krusell–Smith with a network in place of the forecast rule H .**

2.B1 promised that S8 would replace the moments with a network taking the whole histogram as input. This is the session that makes that sentence mean something.

Three Kinds of Learning

Kind	What it is given	What it produces	In this course
supervised	pairs $(\mathbf{x}_i, y_i)$	a map $f_\theta : \mathbf{x} \mapsto \hat{y}$ fitted by minimising a loss	7.A2–7.A3; the Euler-residual solve of 7.B1 is this in disguise
unsupervised	$\mathbf{x}_i$ alone	structure — clusters, factors, a low-dimensional code	the encoder below; embeddings in S9
reinforcement	an environment that returns rewards	a policy, learned by acting	Session 8

Deep learning is not a fourth kind. It is a choice of function class — many composed layers — usable inside any of the three.

The distinction that matters here: supervised learning needs a *target*. 7.B1's trick is that an equilibrium condition supplies one without any data.

The Curse of Dimensionality, Twice

CoD #1 — the state space.

$k \rightarrow \{k, a, h, \dots\}$, and in a heterogeneous-agent model the state includes Γ , which is infinite-dimensional.

Classically: sparse grids and local approximation. Grids are built first, and then used to generate information and pin down parameters.

With a network: **grid-free**. The equilibrium functions are recovered from **sampled** states.

CoD #2 — the unfolding of uncertainty.

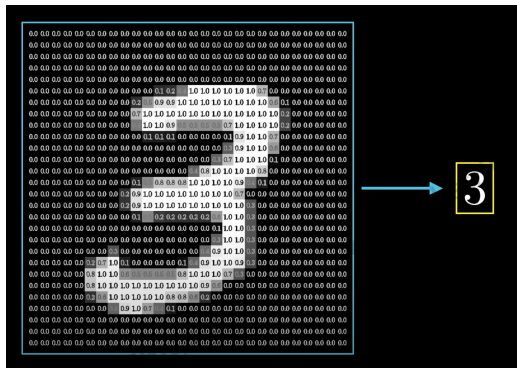
An expectation over future contingencies — and when Γ is a state, even the *space* it lives in is not known in advance.

Classically: discretise and reduce the shock process; impose a Markov structure.

With a network: work on the **ergodic** region, by direct Monte Carlo simulation.

The bottleneck moves. Classical methods spend their budget on *building a grid*; a network spends it on *representing a function*. That is the whole trade, and it scales very differently.

A Handwritten Digit



Everyone in this room reads that instantly.
Write down the rule you used.

You cannot. And that is the point: the mapping from pixels to meaning is one you can **demonstrate** but not **state** — exactly the position we are in with a policy function in twenty state variables.

The next five frames are the whole intuition of this course's use of networks, in a setting where you already know the answer.

The Same Thing, Written Down

The data. $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with $\mathbf{x}_i \in [0, 1]^{784}$ (a 28×28 image, flattened) and $y_i \in \{0, \dots, 9\}$.

The map.

$$f_\theta : \mathbf{x} \mapsto \hat{\mathbf{p}} \in \Delta^9, \quad \hat{y} = \arg \max_k \hat{p}_k,$$

where Δ^9 is the probability simplex on ten classes.

The objective.

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \underbrace{\mathcal{L}(f_\theta(\mathbf{x}_i), y_i)}_{\text{cross-entropy}} + \lambda \|\theta\|_p$$

Look at the shape of it

An objective, a parameter vector, and a search. That is a structural estimation — and θ has no interpretation whatsoever.

The search is stochastic gradient descent and its variants (Adam, RMSProp) — 7.A3, and exactly the *first-order* method 4.A3 said it would have to be. And $\lambda \|\theta\|_p$ is all that stands between 784 inputs and a perfect fit to noise.

Why It Is Possible at All: Structure

A 28×28 image of independent random pixels is essentially never a digit. Handwriting is constrained:

- **anatomically** — muscles, tendons and joints permit only certain strokes, so the patterns are systematic;
- **geometrically** — a '0' has a closed loop, a '1' a vertical stroke, an '8' two stacked loops;
- **spatially** — relative position, size and orientation preserve identity through a great deal of individual variation.

The economist's version

Economic theory constrains the space of plausible models in exactly this way. Structure is what makes a high-dimensional problem **effectively** low-dimensional.

And the constraints can be **learned** rather than hand-built — by data augmentation, by regularisation, and by the choice of architecture (7.A4).

From 784 Pixels to a Handful of Directions

The manifold hypothesis. Real digit images occupy a smooth d -dimensional manifold $\mathcal{M} \subset \mathbb{R}^{784}$ with $d \ll 784$.

In plain terms, only a few writing-style degrees of freedom matter: slant, stroke thickness, whether a loop closes, small translation. The other 780-odd pixel directions are nearly empty.

The macro analogy is exact. Finding a low-dimensional state inside a high-dimensional model is the same problem. A good representation keeps the few variables that matter for the next decision.

You have done this before

Krusell-Smith replaces Γ with its **mean** and finds that almost nothing is lost. That is a one-dimensional manifold hypothesis, asserted rather than learned.

S8's DeepHAM does the learned version: *generalised moments*, chosen by the network instead of by the modeller.

The Encoder View

Split the map in two:

$$\mathbf{x} \xrightarrow{\pi: \mathbb{R}^{784} \rightarrow \mathbb{R}^d} \mathbf{z} \xrightarrow{g_\phi: \mathbb{R}^d \rightarrow \Delta^9} \hat{\mathbf{p}}$$

- π , the **encoder**, flattens the manifold into a latent code $\mathbf{z}$;
- g_ϕ , the **head**, is one or two ordinary layers and a softmax.

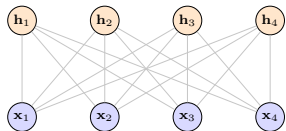
Choices for π : linear — PCA, SVD, random projection;
nonlinear — an autoencoder, t-SNE, UMAP, or the early layers
of a convolutional network.

The translation

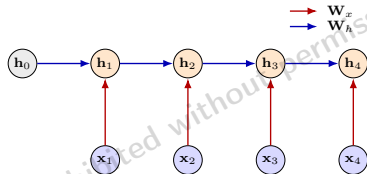
The encoder learns a **nonlinear factor model** instead of imposing a fixed sufficient statistic. Sufficient statistics for prediction — *by learning*.

Rolf et al. (2021) found that for satellite imagery a *random* π plus a ridge regression is often enough (7.A4). The structure is doing the work, not the training.

What the Input's Structure Decides



(a) feedforward: $4 \times 4 = 16$ distinct weights
change the sequence length and $\mathbf{W}$ changes shape



(b) recurrent: only $\mathbf{W}_x$ and $\mathbf{W}_h$
the count does not depend on T

A digit is a **fixed-length** vector, so a feedforward layer is the natural reader: 784 numbers in, one weight each.

Text and a macro time series are not. Order carries information, length varies, and a dense $\mathbf{W}$ sees neither. On the right, one $\mathbf{W}_x$ and one $\mathbf{W}_h$ serve *any* T .

The representation comes first, and the architecture follows from it — which is why the encoder above was worth a frame. 7.A4 returns to this picture and makes the general statement: an architecture is a *constraint* on $\mathbf{W}$.

A Network for Digits

Architecture.

- input: 784 units, one per pixel intensity;
- hidden: several layers, each applying a nonlinear transformation;
- output: 10 units, one per class.

Nonlinearity between the layers — ReLU, sigmoid — with depth and width chosen for the task. 7.A2 shows why the nonlinearity is not optional.

The parallel that matters

The network's structure mirrors the recursive structure of a macro model: **state variables are transformed, layer by layer, into decisions** — exactly as $k_t \mapsto (c_t, l_t)$ through a policy function.

In 7.B1 the input layer is (k, z) and the output layer is c . Nothing else about the object changes.

The Sentence This Session Turns On

Session 5 wrote every approximation the same way:

$$d^n(x) = \sum_i \theta_i \psi_i(x),$$

with $\{\psi_i\}$ — monomials, Chebyshev polynomials, tent functions — **fixed in advance**, and only θ unknown.

A network is

$$\mathcal{N}(x; \theta),$$

and the basis functions are **built out of θ as well**. Nothing else changes: it is still a family, still indexed by a parameter vector, still fitted by making something small.

The claim, precisely

The value of a neural network is **not** that it can approximate anything — so can a polynomial. It is that the basis functions are **chosen by the optimisation** rather than by you.

Which is also the bill: you lose the orthogonality, the conditioning guarantees and the error bounds that made 5.B3 comfortable. 7.A2 counts the cost.

The formulation follows Feng & Miao, *AI for Economists*, ch. 9 §9.7; 5.B1 and 5.B2 promised this frame.

Takeaways, and What 7.A2 Opens

- “Machine learning” means three different jobs — prediction, causation, **computation**. This course is doing the third, and there is no dataset in it.
- The curse of dimensionality bites twice: on the state space, and on the unfolding of uncertainty. A network answers both by **sampling instead of gridding**.
- The bottleneck moves from *constructing a grid* to *representing a function*.
- A network is the projection framework of 5.B1 with **one change**: the basis is learned.

Bridge to 7.A2

Open the box. What is a neuron, what does a layer do, and why is the nonlinearity between them the thing that makes depth mean anything? Then the two theorems: **universal approximation**, and **Baron's rate** — one of which promises much less than it is usually said to.